Phoenix Claims SDK

Version 0.0.1 | Papaya Insurance Infrastructure

Table of Contents

1. Overview
2. Architecture
3. Installation
4. Quick Start
5. Authentication
6. Core Client API Reference
7. React SDK Reference
    - PhoenixProvider
    - PhoenixPortal
    - Hooks
    - Components
    - Events
    - Theming
    - Internationalization
8. Integration Guides
    - Next.js
    - Vite / CRA
    - CORS & Proxy Configuration
9. Type Reference
10. Troubleshooting
11. FAQ
12. Changelog

Overview

The Phoenix Claims SDK provides partner applications with a complete insurance claims portal — claim submission, document upload, claim tracking, and OTP verification — delivered as embeddable React components or a headless Node.js client.

Two packages:

Package	Purpose	Runtime

`@papaya/phoenix`	Core HTTP client + types	Node.js >= 18, browsers
`@papaya/phoenix-react`	React provider, hooks, and UI components	React 18 or 19

The React SDK wraps the core client. You can use the core client alone for server-side integrations or custom UIs, or use the React SDK for a drop-in portal experience.

Architecture

```
┌──────────────────────────────────────────────────────────┐
│  Partner Application (Next.js, Vite, CRA, etc.)  │
│                                                          │
│   ┌──────────────────────────────────────────────┐   │
│   │  @papaya/phoenix-react                       │   │
│   │   ┌──────────┐ ┌──────────┐ ┌──────────────┐ │   │
│   │   │ Provider │ │  Hooks   │ │ Components   │ │   │
│   │   │ (Auth +  │ │ useClaims│ │ ClaimsList   │ │   │
│   │   │  Context) │ │ useClaim │ │ ClaimDetail  │ │   │
│   │   └──────┬───┘ └──────┬───┘ │ Submission   │ │   │
│   │       │            │       │ Portal       │ │   │
│   │       └────────────┘       └──────────────┘ │   │
│   │              │                              │   │
│   │   ┌──────────▼──────────────────────────┐   │   │
│   │   │  @papaya/phoenix (Core Client)      │   │   │
│   │   │  HTTP Client + Types                │   │   │
│   │   └──────────────────────────────────┘   │   │
│   └──────────────────────────────────────────┘   │
│                  │                                       │
└──────────────────────────────────────────────────────────┘
                   │ HTTPS
                   ▼
       Phoenix Claims API
       (phoenix.papaya.asia)
```

Dependency Chain

```
@papaya/phoenix          (zero dependencies, core HTTP client)
  └── @papaya/phoenix-react  (peer: react ^18 || ^19)
```

Installation

The SDK is published to **AWS CodeArtifact** (private npm registry). You must authenticate with AWS before installing.

1. Prerequisites

- **AWS CLI v2** installed (install guide)
- AWS credentials with permission to read from the CodeArtifact repository

Registry details:

Field	Value
Domain	`papaya`
Repository	`sdks`
Region	`ap-southeast-1`
Account	`812652266901`
Endpoint	`https://papaya-812652266901.d.codeartifact.ap-southeast-1.amazonaws.com/npm/sdks/`

2. Authenticate npm/bun with CodeArtifact

Run this command to configure your npm client. The token is valid for 12 hours by default.

```
aws codeartifact login \
  --tool npm \
  --domain papaya \
  --domain-owner 812652266901 \
  --repository sdks \
  --region ap-southeast-1
```

This automatically writes the registry URL and auth token to your `~/.npmrc`. Bun reads `~/.npmrc` natively — no additional configuration needed.

> ***Tip:*** *For CI/CD, use the token export approach instead:*

```
export CODEARTIFACT_AUTH_TOKEN=$(aws codeartifact get-authorization-token \
  --domain papaya \
  --domain-owner 812652266901 \
  --region ap-southeast-1 \
  --query authorizationToken \
  --output text)

npm config set //papaya-812652266901.d.codeartifact.ap-southeast-1.amazonaws.com/npm/sdks/:_authToken=$CODEARTIFACT_AUTH_TOKEN
```

3. Install

```
# Using bun
bun add @papaya/phoenix @papaya/phoenix-react

# Using npm
npm install @papaya/phoenix @papaya/phoenix-react

# Using yarn
yarn add @papaya/phoenix @papaya/phoenix-react
```

If you only need the headless client (no React components):

```
bun add @papaya/phoenix
```

Requirements

- **Node.js** >= 18 (uses native `fetch`)
- **React** 18.x or 19.x (for `@papaya/phoenix-react`)
- **TypeScript** 5.x (recommended, fully typed)

Quick Start

Option A: Drop-in Portal (Fastest)

The `PhoenixPortal` component provides a complete claims experience with zero configuration beyond authentication:

```
import { PhoenixPortal } from '@papaya/phoenix-react';

function App() {
  return (
    <PhoenixPortal
      baseUrl="https://phoenix.papaya.asia"
      policyNumbers={['POL-001', 'POL-002']}
      tenantId="your-tenant-id"
      onClaimSubmitted={(claim) => {
        console.log('New claim:', claim.claimNumber);
      }}
    />
  );
}
```

This renders the full portal: claims list, claim detail, claim submission with document upload and OTP verification. Navigation between views is handled internally.

Option B: Individual Components (Flexible)

For custom layouts and routing, use `PhoenixProvider` with individual components:

```
import {
  PhoenixProvider,
  ClaimsList,
  ClaimDetail,
  ClaimSubmission,
} from '@papaya/phoenix-react';

function ClaimsApp() {
  return (
    <PhoenixProvider
```

```
      config={{ baseUrl: 'https://phoenix.papaya.asia' }}
      policyNumbers={['POL-001']}
      tenantId="your-tenant-id"
    >
      {/* Use your own router to switch between these */}
      <ClaimsList
        onClaimSelect={(claim) => navigate(`/claims/${claim.id}`)}
        onSubmitNew={() => navigate('/submit')}
      />
    </PhoenixProvider>
  );
}
```

Option C: Headless Client (Full Control)

```
import { PhoenixClient } from '@papaya/phoenix';

const client = new PhoenixClient({
  baseUrl: 'https://phoenix.papaya.asia',
  timeout: 30000,
});

// Authenticate
const results = await client.login(['POL-001']);
if (results[0].success && results[0].token) {
  client.setToken(results[0].token);
}

// Fetch claims
const claims = await client.listClaims();
console.log(claims);
```

Authentication

Phoenix SDK uses **policy number authentication**. Policyholders authenticate by providing their policy numbers — the API validates them against the insurance backend and returns JWT tokens.

How It Works

1. The partner app provides one or more policy numbers
2. The SDK calls `POST /auth/phoenix/login` with those numbers
3. The API returns a `LoginResult[]` — one entry per policy number, each with a JWT token if successful
4. The SDK stores tokens internally and attaches them as `Authorization: Bearer <token>` on all subsequent API calls

Multi-Policy Support

A single user may have multiple policies. The SDK authenticates all of them simultaneously and allows switching between policies:

```
<PhoenixProvider
  config={{ baseUrl: '' }}
  policyNumbers={['POL-001', 'POL-002', 'POL-003']}
  tenantId="your-tenant-id"
>
  <PolicySwitcher />
</PhoenixProvider>
```

```
function PolicySwitcher() {
  const { policies, activePolicy, switchPolicy } = usePhoenix();

  return (
    <select
      value={activePolicy?.policyNumber}
      onChange={(e) => switchPolicy(e.target.value)}
    >
      {policies.map((p) => (
        <option key={p.policyNumber} value={p.policyNumber}>
          {p.insuredName} — {p.policyNumber}
        </option>
      ))}
    </select>
  );
}
```

Token Refresh

```
const { token } = await client.refreshToken();
client.setToken(token);
```

Tenant Identification

If the API requires tenant-scoping, pass `tenantId`. It is sent as the `x-tenant-id` HTTP header on every request.

Core Client API Reference

`PhoenixClient`

```
import { PhoenixClient } from '@papaya/phoenix';

const client = new PhoenixClient(config: PhoenixConfig);
```

Constructor

Parameter	Type	Default	Description
`config.baseUrl`	`string`	—	Base URL of the Phoenix API
`config.timeout`	`number`	`30000`	Request timeout in milliseconds

Methods

`setToken(token: string): void`

Set the JWT bearer token for authenticated requests.

`setTenantId(tenantId: string): void`

Set the tenant ID header for multi-tenant environments.

`login(policyNumbers: string[]): Promise<LoginResult[]>`

Authenticate with one or more policy numbers. Returns authentication results including JWT tokens for each valid policy.

```
const results = await client.login(['POL-001', 'POL-002']);
// [
//   { policyNumber: 'POL-001', success: true, token: 'eyJ...',
policy: {...} },
//   { policyNumber: 'POL-002', success: false, message: 'Policy not
found' }
// ]
```

`refreshToken(): Promise<{ token: string }>`

Refresh the current JWT token. Requires an existing valid token.

`listClaims(): Promise<Claim[]>`

Fetch all claims for the authenticated policy.

`getClaim(claimId: string): Promise<ClaimDetail>`

Fetch a single claim with full details, including documents and notes.

`submitClaim(data: CreateClaimInput): Promise<Claim>`

Submit a new insurance claim.

```
const claim = await client.submitClaim({
  claimantName: 'John Doe',
  amountClaimed: 5000000,
  currency: 'VND',
  dateOfLoss: '2026-01-15',
  dateOfService: '2026-01-16',
  providerName: 'City Hospital',
});
```

`uploadDocument(claimId: string, data: UploadDocumentInput): Promise<UploadDocumentResult>`

Upload a document to a claim. Returns a pre-signed upload URL and the document record.

```
const { uploadUrl, document } = await client.uploadDocument(claimId, {
  fileName: 'medical-report.pdf',
```

```
  fileType: 'application/pdf',
  documentType: 'medical_report',
});

// Upload the actual file to the pre-signed URL
await fetch(uploadUrl, { method: 'PUT', body: fileBlob });
```

`getClaimDocuments(claimId: string): Promise<ClaimDocument[]>`

List all documents attached to a claim.

`deleteDocument(claimId: string, documentId: string): Promise<{ success: boolean }>`

Delete a document from a claim.

`requestOtp(claimId: string): Promise<{ success: boolean }>`

Request an OTP verification code for claim submission.

`verifyOtp(claimId: string, code: string): Promise<{ success: boolean; verified: boolean }>`

Verify an OTP code to complete claim submission.

```
await client.requestOtp(claimId);
// User receives OTP via SMS/email

const result = await client.verifyOtp(claimId, '123456');
if (result.verified) {
  console.log('Claim verified successfully');
}
```

React SDK Reference

PhoenixProvider

The context provider that initializes the SDK client, manages authentication state, and provides context to all child components.

```
import { PhoenixProvider } from '@papaya/phoenix-react';

<PhoenixProvider
  config={{ baseUrl: 'https://phoenix.papaya.asia' }}
  tenantId="your-tenant-id"
  policyNumbers={['POL-001']}
  theme={customTheme}
  locale="vi"
>
  {children}
</PhoenixProvider>
```

Props

Prop	Type	Default	Description
`config`	`PhoenixConfig`	—	Client configuration (`baseUrl` , `timeout`)
`tenantId`	`string`	—	Tenant identifier (sent as `x-tenant-id` header)
`policyNumbers`	`string[]`	—	Policy numbers for auto-login on mount
`theme`	`PhoenixTheme`	`defaultTheme`	Custom color and typography theme
`locale`	`Locale`	`'en'`	Display language (`'en'` or `'vi'`)
`children`	`ReactNode`	—	Child components

Context Value (`usePhoenix()`)

```
const {
  client,          // PhoenixClient instance
  events,          // PhoenixEventEmitter instance
  policies,        // PolicyInfo[] — all authenticated policies
  activePolicy,    // PolicyInfo | null — currently selected policy
  isAuthenticated, // boolean
  loading,         // boolean — true during login
  locale,          // Locale
  login,           // (policyNumbers: string[]) =>
Promise<LoginResult[]>
  switchPolicy,    // (policyNumber: string) => void
```

```
  logout,          // () => void
} = usePhoenix();
```

> **Note:** `usePhoenix()` *must be called inside a* `<PhoenixProvider>`*. Calling it outside throws:* `"usePhoenix must be used within a <PhoenixProvider>"`

PhoenixPortal

A self-contained portal that wraps `PhoenixProvider` with its own internal navigation. Ideal for embedding a complete claims experience in a single component.

```
import { PhoenixPortal } from '@papaya/phoenix-react';

<PhoenixPortal
  baseUrl="https://phoenix.papaya.asia"
  policyNumbers={['POL-001']}
  tenantId="your-tenant-id"
  theme={customTheme}
  locale="en"
  onClaimSubmitted={(claim) => console.log(claim)}
  className="my-portal"
/>
```

Props

Prop	Type	Default	Description
`baseUrl`	`string`	—	Phoenix API base URL
`policyNumbers`	`string[]`	—	Policy numbers for authentication
`tenantId`	`string`	—	Tenant identifier
`theme`	`PhoenixTheme`	`defaultTheme`	Custom theme
`locale`	`Locale`	`'en'`	Display language
`onClaimSubmitted`	`(claim: { id: string; claimNumber: string }) => void`	—	Callback when a claim is successfully submitted

`className`	`string`	—	CSS class for the portal container

Internal Views

The portal manages four views internally:

1. **List** — Shows all claims with status badges
2. **Detail** — Single claim with documents, notes, and amounts
3. **Submit** — Multi-step claim submission form
4. **Additional Docs** — Upload additional documents for existing claims

---

Hooks

`useClaims()`

Fetches all claims for the authenticated policy. Auto-fetches on mount and when authentication changes.

```
const { data, loading, error, refetch } = useClaims();
```

Return	Type	Description
`data`	`Claim[]`	Array of claims (empty while loading)
`loading`	`boolean`	True during fetch
`error`	`Error | null`	Fetch error, if any
`refetch`	`() => Promise<void>`	Manually re-fetch claims

`useClaim(claimId: string)`

Fetches a single claim with full details.

```
const { data, loading, error } = useClaim('claim-uuid');
```

Return	Type	Description
`data`	`ClaimDetail | null`	Claim detail with documents and notes
`loading`	`boolean`	True during fetch

`error`	`Error | null`	Fetch error, if any

`usePhoenixEvent(event, listener)`

Subscribe to SDK lifecycle events with automatic cleanup.

```
import { usePhoenixEvent } from '@papaya/phoenix-react';

usePhoenixEvent('claim:created', (payload) => {
  console.log('New claim:', payload.claimNumber);
  analytics.track('claim_submitted', payload);
});
```

See Events for all available event types.

Components

`ClaimsList`

Renders a list of claim cards with status badges, amounts, and dates.

```
<ClaimsList
  onClaimSelect={(claim) => navigate(`/claims/${claim.id}`)}
  onSubmitNew={() => navigate('/submit')}
  locale="en"
/>
```

Prop	Type	Description
`onClaimSelect`	`(claim: Claim) => void`	Called when a claim card is tapped
`onSubmitNew`	`() => void`	Called when "Submit New Claim" is tapped
`locale`	`Locale`	Override the provider's locale

States: Loading spinner, error with retry button, empty state with CTA, claim cards.

`ClaimDetail`

Renders a single claim with amounts, dates, documents, and notes in collapsible sections.

```
<ClaimDetail
  claimId="claim-uuid"
  onBack={() => navigate('/claims')}
  onAdditionalDocs={(claimId) => navigate(`/claims/${claimId}/docs`)}
  locale="en"
/>
```

Prop	Type	Description
`claimId`	`string`	The claim UUID to display
`onBack`	`() => void`	Called when back button is tapped
`onAdditionalDocs`	`(claimId: string) => void`	Called when "Submit Additional Documents" is tapped
`locale`	`Locale`	Override the provider's locale

Features: Collapsible documents and notes sections. Shows "Submit Additional Documents" button when claim status is `additional_docs_required`.

`ClaimSubmission`

Multi-step form for submitting a new claim: Information, Documents, Review, OTP Verification, Complete.

```
<ClaimSubmission
  onComplete={(claim) => navigate(`/claims/${claim.id}`)}
  onCancel={() => navigate('/claims')}
  locale="en"
/>
```

Prop	Type	Description
`onComplete`	`(claim: { id: string; claimNumber: string }) => void`	Called after OTP verification succeeds
`onCancel`	`() => void`	Called when cancel button is tapped
`locale`	`Locale`	Override the provider's locale

Steps:

Step	Description
1. Claim Information	Claimant name, amount, currency, dates, provider
2. Upload Documents	Select document type, attach files
3. Review & Submit	Summary of all entered data, submit button
4. OTP Verification	Enter OTP code sent to claimant
5. Complete	Success confirmation with claim number

Supported document types: Medical Report, Invoice, Receipt, ID Card, Prescription, Discharge Summary, Claim Form, Other.

Accepted file formats: Images (`image/*`), PDF (`.pdf`), Word (`.doc` , `.docx`).

`AdditionalDocs`

Upload additional documents to an existing claim (e.g., when the insurer requests more documentation).

```
<AdditionalDocs
  claimId="claim-uuid"
  onComplete={() => navigate('/claims')}
  onBack={() => navigate(-1)}
  locale="en"
/>
```

Prop	Type	Description
`claimId`	`string`	The claim UUID to upload documents to
`onComplete`	`() => void`	Called after all documents are uploaded
`onBack`	`() => void`	Called when back button is tapped
`locale`	`Locale`	Override the provider's locale

`StatusBadge`

Color-coded pill badge for claim statuses.

```
<StatusBadge status="approved" locale="en" />
```

Prop	Type	Description
`status`	`string`	Claim status string
`locale`	`Locale`	Display language (default: `'en'`)

Supported statuses and their colors:

Status	English Label	Color
`submitted`	Submitted	Blue
`pending_review`	Pending Review	Amber
`under_review`	Under Review	Orange
`ai_processing`	Processing	Purple
`adjudicated`	Adjudicated	Indigo
`approved`	Approved	Green
`partially_approved`	Partially Approved	Lime
`denied`	Denied	Red
`appealed`	Appealed	Amber
`settled`	Settled	Emerald
`closed`	Closed	Gray
`additional_docs_required`	Docs Required	Orange

Events

The SDK emits typed events throughout the claim lifecycle. Use these for analytics, logging, or triggering side effects.

Event Types

Event	Payload	When
`claim:creating`	`{ claimantName, amountClaimed }`	Claim submission started
`claim:created`	`{ claimId, claimNumber }`	Claim successfully created on server

`claim:creation_failed`	`{ error }`	Claim submission failed
`claim:cancelled`	`{}`	User cancelled claim submission
`claim:document_uploaded`	`{ claimId, fileName, documentType? }`	Document uploaded successfully
`claim:document_upload_failed`	`{ claimId, fileName, error }`	Document upload failed
`claim:otp_requested`	`{ claimId }`	OTP verification code requested
`claim:otp_verified`	`{ claimId }`	OTP verified successfully
`claim:otp_failed`	`{ claimId, error }`	OTP verification failed

Using Events in React

```
import { usePhoenixEvent } from '@papaya/phoenix-react';

function Analytics() {
  usePhoenixEvent('claim:created', (payload) => {
    gtag('event', 'claim_submitted', {
      claim_id: payload.claimId,
      claim_number: payload.claimNumber,
    });
  });

  usePhoenixEvent('claim:creation_failed', (payload) => {
    Sentry.captureMessage('Claim submission failed', {
      extra: { error: payload.error },
    });
  });

  return null;
}
```

Using Events Imperatively

```
const { events } = usePhoenix();
```

```
// Subscribe
const unsubscribe = events.on('claim:created', (payload) => {
  console.log('Created:', payload);
});

// Later: unsubscribe
unsubscribe();
```

Theming

Components are styled with CSS custom properties scoped under `[data-phoenix-root]`. Override the default theme by passing a `PhoenixTheme` object.

Theme Interface

```
interface PhoenixTheme {
  colors: {
    primary: string;        // Primary actions, buttons, links
    primaryHover: string;   // Primary hover state
    success: string;        // Success states (approved)
    warning: string;        // Warning states
    error: string;          // Error states (denied, failures)
    textPrimary: string;    // Main body text
    textSecondary: string;  // Secondary/label text
    textMuted: string;      // Muted/caption text
    background: string;     // Page background
    surface: string;        // Card/section background
    border: string;         // Border color
  };
  fontFamily: string;       // CSS font-family string
  borderRadius: string;     // Card border radius (e.g., '12px')
}
```

Default Theme

```
import { defaultTheme } from '@papaya/phoenix-react';

// defaultTheme values:
```

```
{
  colors: {
    primary: '#E30613',        // Papaya red
    primaryHover: '#B8050F',
    success: '#16a34a',
    warning: '#d97706',
    error: '#dc2626',
    textPrimary: '#111827',
    textSecondary: '#6b7280',
    textMuted: '#9ca3af',
    background: '#f9fafb',
    surface: '#ffffff',
    border: '#e5e7eb',
  },
  fontFamily: '-apple-system, BlinkMacSystemFont, "Segoe UI", Roboto,
...',
  borderRadius: '12px',
}
```

Custom Theme Example

```
const partnerTheme: PhoenixTheme = {
  colors: {
    ...defaultTheme.colors,
    primary: '#0066CC',       // Your brand color
    primaryHover: '#004C99',
  },
  fontFamily: '"Inter", sans-serif',
  borderRadius: '8px',
};

<PhoenixProvider
  config={{ baseUrl: '' }}
  policyNumbers={['POL-001']}
  theme={partnerTheme}
>
  {children}
</PhoenixProvider>
```

CSS Custom Properties

The theme generates these CSS variables on `[data-phoenix-root]`:

Variable	Maps to
`--phoenix-color-primary`	`colors.primary`
`--phoenix-color-primary-hover`	`colors.primaryHover`
`--phoenix-color-success`	`colors.success`
`--phoenix-color-warning`	`colors.warning`
`--phoenix-color-error`	`colors.error`
`--phoenix-color-text-primary`	`colors.textPrimary`
`--phoenix-color-text-secondary`	`colors.textSecondary`
`--phoenix-color-text-muted`	`colors.textMuted`
`--phoenix-color-background`	`colors.background`
`--phoenix-color-surface`	`colors.surface`
`--phoenix-color-border`	`colors.border`
`--phoenix-font-family`	`fontFamily`
`--phoenix-border-radius`	`borderRadius`

You can also override these in your own CSS:

```
[data-phoenix-root] {
  --phoenix-color-primary: #0066CC;
  --phoenix-border-radius: 8px;
}
```

Internationalization

The SDK ships with English (`en`) and Vietnamese (`vi`) translations. Set the locale on the provider or individual components.

Setting the Locale

```
// Provider-level (applies to all children)
<PhoenixProvider locale="vi" ...>

// Component-level override
<ClaimsList locale="en" />
```

Translation Functions

```
import { t, getStatusLabel, getDocTypeLabel } from '@papaya/phoenix-
react';

t('en', 'claims.title');              // "My Claims"
t('vi', 'claims.title');              // "Yeu cau boi thuong"

getStatusLabel('en', 'approved');     // "Approved"
getStatusLabel('vi', 'approved');     // "Da duyet"

getDocTypeLabel('en', 'invoice');     // "Invoice"
getDocTypeLabel('vi', 'invoice');     // "Hoa don"
```

Available Translation Keys

Claims List: `claims.title`, `claims.empty`, `claims.empty_desc`, `claims.submit_new`, `claims.loading`, `claims.error`, `claims.retry`, `claims.submitted_date`

Claim Detail: `detail.title`, `detail.back`, `detail.amount_claimed`, `detail.amount_approved`, `detail.amount_paid`, `detail.date_of_loss`, `detail.date_of_service`, `detail.provider`, `detail.documents`, `detail.documents_empty`, `detail.notes`, `detail.additional_docs`, `detail.loading`, `detail.error`

Submission: `submit.title`, `submit.step_info`, `submit.step_documents`, `submit.step_review`, `submit.step_otp`, `submit.step_complete`, `submit.claimant_name`, `submit.amount_claimed`, `submit.currency`, `submit.date_of_loss`, `submit.date_of_service`, `submit.provider_name`, `submit.next`, `submit.back`, `submit.cancel`, `submit.submit`, `submit.uploading`, `submit.upload_doc`, `submit.choose_file`,

`submit.doc_type`, `submit.remove`, `submit.review_info`, `submit.otp_sent`, `submit.otp_code`, `submit.verify`, `submit.resend`, `submit.success_title`, `submit.success_desc`, `submit.view_claim`, `submit.done`

Additional Docs: `additional.title`, `additional.desc`, `additional.submit`

Integration Guides

Next.js Integration

1. Install packages

```
bun add @papaya/phoenix @papaya/phoenix-react
```

2. Configure `next.config.ts`

The SDK client runs in the browser. To avoid CORS issues during development, use Next.js rewrites to proxy API requests:

```
// next.config.ts
import type { NextConfig } from 'next';

const PHOENIX_API =
  process.env.NEXT_PUBLIC_PHOENIX_URL ??
'https://phoenix.papaya.asia';

const nextConfig: NextConfig = {
  transpilePackages: ['@papaya/phoenix', '@papaya/phoenix-react'],
  async rewrites() {
    return [
      {
        source: '/auth/:path*',
        destination: `${PHOENIX_API}/auth/:path*`,
      },
    ];
  },
```

```
};

export default nextConfig;
```

3. Create a config file

```
// src/lib/config.ts

// Empty baseUrl = same-origin requests, proxied by Next.js rewrites
export const PHOENIX_URL = process.env.NEXT_PUBLIC_PHOENIX_URL ?? '';
export const POLICY_NUMBERS =
  (process.env.NEXT_PUBLIC_POLICY_NUMBERS ?? '').split(',');
export const TENANT_ID = process.env.NEXT_PUBLIC_TENANT_ID ??
undefined;
```

4. Create a provider wrapper

```
// src/providers/PhoenixSetup.tsx
'use client';

import { PhoenixProvider } from '@papaya/phoenix-react';
import { PHOENIX_URL, POLICY_NUMBERS, TENANT_ID } from '@/lib/config';

export function PhoenixSetup({ children }: { children: React.ReactNode
}) {
  return (
    <PhoenixProvider
      config={{ baseUrl: PHOENIX_URL }}
      tenantId={TENANT_ID}
      policyNumbers={POLICY_NUMBERS}
      locale="en"
    >
      {children}
    </PhoenixProvider>
  );
}
```

5. Use components in pages

```
// src/app/claims/page.tsx
'use client';

import { ClaimsList } from '@papaya/phoenix-react';
import { PhoenixSetup } from '@/providers/PhoenixSetup';

export default function ClaimsPage() {
  return (
    <PhoenixSetup>
      <ClaimsList
        onClaimSelect={(claim) => window.location.href =
`/claims/${claim.id}`}
        onSubmitNew={() => window.location.href = '/submit'}
      />
    </PhoenixSetup>
  );
}
```

> ***Important:*** *All SDK components use* `'use client'` *— they cannot be server components because they use React hooks and browser APIs.*

Vite / Create React App

```
// main.tsx
import { PhoenixPortal } from '@papaya/phoenix-react';

function App() {
  return (
    <PhoenixPortal
      baseUrl="https://phoenix.papaya.asia"
      policyNumbers={['POL-001']}
      tenantId="your-tenant-id"
    />
  );
}
```

For Vite, no special configuration is needed. The SDK uses standard ESM imports and inline styles (no external CSS to configure).

CORS & Proxy Configuration

The SDK's HTTP client uses browser `fetch`. When running on a different origin than the API, you must handle CORS.

Development (Recommended: Proxy)

Use your framework's dev server proxy to route API requests through the same origin:

Next.js — Use `rewrites()` in `next.config.ts` (shown above)

Vite — Use `server.proxy` in `vite.config.ts`:

```
// vite.config.ts
export default defineConfig({
  server: {
    proxy: {
      '/auth': {
        target: 'https://phoenix.papaya.asia',
        changeOrigin: true,
      },
    },
  },
});
```

When using a proxy, set `baseUrl` to `''` (empty string) so requests are made to the same origin.

Production

In production, either:

1. **Deploy behind the same domain** — API and frontend on the same origin (no CORS needed)
2. **CORS headers on the API** — The Phoenix API must include your domain in its `Access-Control-Allow-Origin` header
3. **Reverse proxy** — Use your CDN/load balancer (CloudFront, nginx) to route `/auth/*` to the API

Type Reference

Configuration

```
interface PhoenixConfig {
  baseUrl: string;
  timeout?: number;  // Default: 30000ms
}
```

Authentication

```
interface LoginResult {
  policyNumber: string;
  success: boolean;
  message?: string;      // Error message when success is false
  token?: string;        // JWT token when success is true
  policy?: PolicyInfo;   // Policy details when success is true
}

interface PolicyInfo {
  id: string;
  policyNumber: string;
  insuredName: string;
  status: string;
}
```

Claims

```
interface Claim {
  id: string;
  claimNumber: string;
  status: string;
  claimantName: string;
  providerName: string | null;
  amountClaimed: number;
  amountApproved: number | null;
  amountPaid: number | null;
  currency: string;
```

```
  dateOfLoss: string | null;
  dateOfService: string | null;
  createdAt: string;
}

interface ClaimDetail extends Claim {
  documents: ClaimDocument[];
  notes: ClaimNote[];
  aiSummary: string | null;
  aiRecommendation: string | null;
}

interface CreateClaimInput {
  claimantName: string;
  amountClaimed: number;
  currency?: string;        // Default: 'VND'
  dateOfLoss?: string;      // ISO date string
  dateOfService?: string;   // ISO date string
  providerName?: string;
}
```

Documents

```
interface ClaimDocument {
  id: string;
  fileName: string;
  fileType: string | null;
  fileUrl: string;
  fileSizeBytes: number | null;
  documentType: string | null;
  createdAt: string;
}

interface UploadDocumentInput {
  fileName: string;
  fileType: string;            // MIME type
  documentType?: string;       // e.g., 'medical_report', 'invoice'
}

interface UploadDocumentResult {
  uploadUrl: string;           // Pre-signed URL to PUT the file
```

```
  document: ClaimDocument;     // Created document record
}
```

Notes

```
interface ClaimNote {
  id: string;
  content: string;
  noteType: string;
  agentName: string | null;
  createdAt: string;
}
```

Troubleshooting

npm Install Fails: "401 Unauthorized" or "404 Not Found"

Symptom: `npm install` or `bun add` fails with `401 Unauthorized`, `403 Forbidden`, or `404 Not Found` when installing `@papaya/phoenix`.

Cause: The SDK is hosted on AWS CodeArtifact (private registry). Your npm client is not authenticated or the token has expired.

Fix:

1. Re-run the login command (tokens expire after 12 hours):

```
aws codeartifact login \
  --tool npm \
  --domain papaya \
  --domain-owner 812652266901 \
  --repository sdks \
  --region ap-southeast-1
```

2. Verify your AWS credentials are valid: `aws sts get-caller-identity`

3. Verify you have `codeartifact:GetAuthorizationToken` and `codeartifact:ReadFromRepository` permissions
4. Check `~/.npmrc` contains a line pointing to `papaya-812652266901.d.codeartifact.ap-southeast-1.amazonaws.com`
5. For CI/CD, ensure the auth token is refreshed before each `npm install` step

---

CORS Error: "Failed to fetch"

Symptom: Browser console shows `TypeError: Failed to fetch` or `Access to fetch has been blocked by CORS policy`.

Cause: The SDK client runs in the browser and makes requests to a different origin (e.g., `localhost:3000` calling `phoenix.papaya.asia`).

Solutions:

1. **Use a dev proxy** (recommended for development):

   ```
   // next.config.ts
   async rewrites() {
     return [{ source: '/auth/:path*', destination:
   'https://phoenix.papaya.asia/auth/:path*' }];
   }
   ```

   Then set `baseUrl` to `''` (empty string).

2. **Use the same origin** in production — deploy your app on the same domain as the API.
3. **Configure CORS on the API** — Ensure the API includes your domain in `Access-Control-Allow-Origin`.

Authentication Fails Silently

Symptom: `isAuthenticated` remains `false` after mount. No error shown.

Checks:

1. Verify `policyNumbers` is a non-empty array of valid policy numbers
2. Check the browser Network tab — look for `POST /auth/phoenix/login` and inspect the response
3. Each item in the response has a `success` field — check if any return `false` with a `message`
4. Verify `baseUrl` is correct and reachable
5. If using a proxy, verify the rewrite rule is working (check that requests reach the API)

Debug:

```
const { loading, isAuthenticated, policies } = usePhoenix();
console.log({ loading, isAuthenticated, policies });
```

"usePhoenix must be used within a PhoenixProvider"

Cause: A component calling `usePhoenix()`, `useClaims()`, `useClaim()`, or `usePhoenixEvent()` is rendered outside of a `<PhoenixProvider>`.

Fix: Ensure all SDK hook consumers are descendants of `<PhoenixProvider>`:

```
// Wrong
<div>
  <PhoenixProvider ...>...</PhoenixProvider>
  <ClaimsList />  {/* This is OUTSIDE the provider */}
</div>

// Correct
<PhoenixProvider ...>
  <ClaimsList />  {/* This is INSIDE the provider */}
</PhoenixProvider>
```

> ***Note:*** *`<PhoenixPortal>` includes its own `<PhoenixProvider>` internally, so components used inside the portal don't need an extra provider.*

Components Not Rendering / Blank Screen

Checks:

1. **Provider is loading:** The SDK auto-logs in on mount. During this time, `loading` is `true` and components may show a spinner. Wait for authentication to complete.
2. **No policy numbers:** If `policyNumbers` is empty or undefined, no login occurs and components that depend on authentication will show nothing.
3. **Network error:** Open the Network tab and check for failed requests.
4. **Build error with transpilePackages:** For Next.js, ensure both packages are in `transpilePackages`:

```
transpilePackages: ['@papaya/phoenix', '@papaya/phoenix-react'],
```

Request Timeout

Symptom: Requests hang and then fail with an abort error.

Cause: The default timeout is 30 seconds. Slow networks or large file uploads may exceed this.

Fix:

```
<PhoenixProvider
  config={{ baseUrl: '', timeout: 60000 }}  // 60 seconds
  ...
>
```

Or on the client directly:

```
const client = new PhoenixClient({ baseUrl: '', timeout: 60000 });
```

OTP Not Received

Checks:

1. The `requestOtp` call must succeed — check the Network tab for `POST /auth/phoenix/claims/{id}/otp/request`
2. OTP is sent via the channel configured on the API side (SMS, email) — verify the claimant's contact info is correct
3. The `claim:otp_requested` event fires if the request succeeded — listen for `claim:otp_failed` for errors
4. Wait 30 seconds before resending (rate limiting may apply)

Document Upload Fails

Symptom: Document upload succeeds in the SDK (no error) but the file is not actually uploaded.

Explanation: The SDK's `uploadDocument` method requests a pre-signed URL from the API and creates the document record. The **actual file upload** to the pre-signed URL is a separate step handled by the `ClaimSubmission` and `AdditionalDocs` components internally.

If building a custom UI, you must upload the file yourself:

```
const { uploadUrl } = await client.uploadDocument(claimId, {
  fileName: 'report.pdf',
  fileType: 'application/pdf',
  documentType: 'medical_report',
});

// You must PUT the file to the pre-signed URL
await fetch(uploadUrl, {
  method: 'PUT',
  body: file,
  headers: { 'Content-Type': file.type },
});
```

Styles Conflict with Host App

Symptom: SDK components look broken or inherit unexpected styles from the host application.

Explanation: The SDK uses inline styles and CSS custom properties scoped under `[data-phoenix-root]`. It also injects `box-sizing: border-box` on all its elements.

If host CSS bleeds in:

1. The SDK sets `fontFamily` from the theme — if it looks wrong, set `fontFamily` explicitly in your theme
2. CSS resets that target `*` or `button` elements may affect SDK components — scope your resets
3. Tailwind's `preflight` base styles may interfere — if so, add `[data-phoenix-root]` to your Tailwind prefix exclusion

---

Mobile: iOS Auto-Zoom on Input Focus

Symptom: On iOS Safari, the page zooms in when a form input is focused.

Explanation: iOS Safari zooms in on any input with `font-size` less than 16px. The SDK uses `font-size: 16px` on all inputs to prevent this.

If you override input styles and set a smaller font size, the zoom will return. To prevent it at the viewport level:

```
<meta name="viewport" content="width=device-width, initial-scale=1,
maximum-scale=1" />
```

Responsive Layout Issues

Symptom: Components don't adapt to small screens.

Explanation: The SDK injects responsive CSS for screens narrower than 480px:

- Two-column grids collapse to single column (`.phoenix-grid-2`)
- Document upload rows wrap vertically (`.phoenix-row-wrap`)
- Step labels in the submission stepper are hidden (`.phoenix-step-label`)

These styles are injected via a `<style>` tag from `<PhoenixProvider>`. If your host app uses Shadow DOM or `<iframe>` isolation, the injected styles may not apply.

FAQ

Q: Can I use the SDK without React? A: Yes. `@papaya/phoenix` is a framework-agnostic HTTP client. Use it in Node.js, Vue, Svelte, or any JavaScript environment with `fetch` support.

Q: Does the SDK bundle any CSS files? A: No. All styles are inline or injected via `<style>` tags. There are no CSS imports to configure in your build.

Q: Can I customize the UI beyond theming? A: Yes. Use the headless hooks (`useClaims` , `useClaim` , `usePhoenix`) with your own components for complete UI control. The pre-built components are for rapid integration.

Q: How do I support additional languages? A: The SDK currently supports English and Vietnamese. For other languages, use the headless hooks and provide your own translations in the UI layer.

Q: What happens if a policy number is invalid? A: The `login` method returns results for all policy numbers. Invalid ones return `{ success: false, message: '...' }`. The SDK authenticates with the first valid policy and ignores invalid ones.

Q: Can I embed the portal in an iframe? A: Yes. `<PhoenixPortal>` works in iframes. Ensure the `baseUrl` is set correctly from the iframe's perspective (use a proxy if needed to avoid CORS).

Q: Is the SDK tree-shakeable? A: Yes. Both packages use ESM and named exports. If you only import `PhoenixPortal` , bundlers will tree-shake unused hooks and utilities.

Changelog

0.0.1 (Initial Release)

- Core client with full claims API surface (login, claims CRUD, documents, OTP)
- React provider with auto-login and multi-policy support
- Pre-built components: ClaimsList, ClaimDetail, ClaimSubmission, AdditionalDocs, PhoenixPortal, StatusBadge
- CSS custom property theming with `defaultTheme`
- Typed event system with 9 lifecycle events
- i18n support for English and Vietnamese
- Mobile-responsive layout with 480px breakpoints